

Welcome to the experiment section



ExperimentNo: 01

Aim: Loading and Exploring a Dataset (e.g., Iris, MNIST) Using Pandas and Calculating Summary Statistics

Objective:

- To learn how to load datasets (Iris and MNIST) into Python using the Pandas library.
- To explore the structure and content of the datasets.
- To calculate and interpret summary statistics (e.g., mean, median, standard deviation) for the datasets.

Prerequisites

- **Software:** Python 3.x installed with the following libraries:
 - pandas
 - numpy
 - scikit-learn (for Iris)
 - tensorflow or keras (for MNIST, optional for raw data loading)
- **Hardware:** A computer with sufficient memory (at least 4GB RAM recommended).
- **Skills:**
 - Basic Python programming knowledge.
 - Familiarity with installing Python libraries using pip (e.g., pip install pandas numpy scikit-learn).
- **Dataset Availability:**
 - Iris dataset (built into scikit-learn).
 - MNIST dataset (available via tensorflow.keras.datasets or as raw CSV files).

Theory

- **Pandas:** A powerful Python library for data manipulation and analysis, providing data structures like DataFrames.
- **Iris Dataset:** A classic multivariate dataset with 150 samples of 4 features (sepal length, sepal width, petal length, petal width) and 3 species classes.
- **MNIST Dataset:** A large database of handwritten digits (0–9) with 70,000 images (28x28 pixels each), often used for machine learning.
- **Summary Statistics:** Descriptive measures (e.g., mean, median, min, max) that summarize the central tendency, dispersion, and shape of a dataset's distribution.

Apparatus/Software Requirements

- Python IDE or Jupyter Notebook.
- Internet connection (for initial library installation).

Procedure

Step 1: Set Up the Environment

1. Install required libraries if not already installed:

```
pip install pandas numpy scikit-learn tensorflow
```

2. Launch Jupyter Notebook or your preferred Python environment.

Step 2: Load the Iris Dataset

1. Import necessary libraries.
2. Load the Iris dataset using scikit-learn.
3. Convert it to a Pandas DataFrame for analysis.

Step 3: Explore the Iris Dataset

1. Display the first few rows of the dataset.
2. Check the dataset's shape, column names, and data types.
3. Calculate summary statistics.

Step 4: Load the MNIST Dataset

1. Import the MNIST dataset using tensorflow.keras.datasets.
2. Preprocess the data (e.g., flatten images) and create a DataFrame.
3. Explore the dataset and calculate summary statistics.

Step 5: Analyze and Interpret Results

1. Compare summary statistics between Iris and MNIST.
2. Document observations (e.g., range of values, missing data).

Program Code

Lab Program: Loading and Exploring Datasets with Pandas

Import libraries

```
import pandas as pd
import numpy as np
from sklearn.datasets import load_iris
from tensorflow.keras.datasets import mnist
```

Step 1: Load and Explore Iris Dataset

```
print("=== Exploring Iris Dataset ===")
```

Load Iris dataset

```
iris = load_iris()
iris_df = pd.DataFrame(data=iris.data, columns=iris.feature_names)
iris_df['target'] = iris.target
```

Display first 5 rows

```
print("First 5 rows of Iris dataset:")
print(iris_df.head())
```

Check dataset info

```
print("\nDataset Info:")
print(iris_df.info())
```

Calculate summary statistics

```
print("\nSummary Statistics for Iris dataset:")
print(iris_df.describe())
```

Step 2: Load and Explore MNIST Dataset

```
print("\n=== Exploring MNIST Dataset ===")
```

Load MNIST dataset

```
(X_train, y_train), (X_test, y_test) = mnist.load_data()
```

Flatten images and create DataFrame (using first 1000 samples for simplicity)

```
mnist_df = pd.DataFrame(X_train.reshape(X_train.shape[0], -1)[:1000])  
mnist_df['label'] = y_train[:1000]
```

Display first 5 rows

```
print("\nFirst 5 rows of MNIST dataset:")  
print(mnist_df.head())
```

Check dataset info

```
print("\nDataset Info:")  
print(mnist_df.info())
```

Calculate summary statistics (for pixel values)

```
print("\nSummary Statistics for MNIST dataset (pixel values):")  
print(mnist_df.drop(columns=['label']).describe())
```

Expected Output

Iris Dataset

First 5 rows of Iris dataset:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

Summary Statistics:

- Mean sepal length: ~5.84 cm
- Min petal width: 0.1 cm
- Max petal length: 6.9 cm

MNIST Dataset

- **First 5 Rows** (partial view of flattened 28x28=784 pixels + label):

First 5 rows of MNIST dataset:

	0	1	2	...	783	label
0	0	0	0	...	0	5
1	0	0	0	...	0	0
2	0	0	0	...	0	4
3	0	0	0	...	0	1
4	0	0	0	...	0	9

Summary Statistics (for pixel values, 0–255 range):

- Mean: ~30–40 (varies by sample subset)
- Min: 0
- Max: 255

Observations

- **Iris:** The dataset has 150 rows and 5 columns (4 features + target). All features are numeric with no missing values. Summary statistics show a range of values typical for flower measurements.
- **MNIST:** The dataset has 60,000 training images (subset to 1000 here), with 784 pixel features per image. Pixel values range from 0 (white) to 255 (black), with labels 0–9.

Conclusion

This lab demonstrated how to load and explore the Iris and MNIST datasets using Pandas. Summary statistics provided insights into the data's distribution, preparing the ground for further analysis (e.g., machine learning).

Viva question:

1. What differences do you notice in the summary statistics between Iris and MNIST?
2. How might missing values or outliers affect the summary statistics?
3. Suggest a way to visualize the Iris dataset's features (e.g., scatter plot).

Experiment No: 02

Machine Learning

Aim:

Split Data into Train/Validation/Test sets (75-15-15) and save using `train_test_split`.

Objective:

- To learn how to partition a dataset into training, validation, and testing subsets.
- To implement data splitting using `sklearn`'s `train_test_split` function.
- To save the resulting subsets into separate CSV files for future use.

Prerequisites:

- Software: Python 3.x installed with `pandas`, `scikit-learn`, `matplotlib`.
- Hardware: A computer with minimum 4GB RAM.
- Skills: Basic Python programming and familiarity with CSV data handling.

Theory:

In machine learning, datasets must be divided into three distinct parts to ensure proper evaluation and avoid overfitting. The training set (75%) is used to fit the model. The validation set (15%) is used for hyperparameter tuning and model selection. The test set (15%) is used to assess the final model's performance on unseen data. The `train_test_split` function from `sklearn` enables easy and reproducible splitting of datasets. Storing these subsets into separate files ensures reproducibility and easier deployment.

Apparatus/Software Requirements

- Python IDE or Jupyter Notebook.
- Internet connection (for initial library installation).

Procedure:

- ◆ Step 1: Load the dataset (e.g., *Cars_Datasets_2025.csv*) into a `pandas` DataFrame using `pd.read_csv()`. Display the first few rows and shape to ensure it is correctly loaded.
- ◆ Step 2: Separate the input features (X) and the target variable (y). For example, X = "Engines" and "CC/Battery Capacity", while y = "Total Speed".
- ◆ Step 3: Apply `train_test_split` to divide the data into two parts: Training (70%) and Temporary (30%). The training set will be used to build the model.
- ◆ Step 4: Further split the Temporary set equally into Validation (15%) and Test (15%) subsets using another `train_test_split`. The validation set will help in tuning the model, while the test set is reserved for final evaluation.

- ◆ Step 5: Concatenate the features (X) and target (y) back into combined DataFrames for Train, Validation, and Test sets, making them easier to handle.
- ◆ Step 6: Save each of the three subsets (train, val, test) into separate CSV files using `to_csv()` for future use.
- ◆ Step 7: Print and verify the shapes of Train, Validation, and Test sets to confirm the correct 75-15-15 ratio split.

Program Code:

```
import pandas as pd
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt

df = pd.read_csv("Cars_Datasets_2025.csv", encoding="ISO-8859-1")
print(df.head())
print(df.shape)

X = df[["Engines", "CC/Battery Capacity"]]
y = df["Total Speed"]

# First split: Train (70%) and Temp (30%)
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.30,
                                                    random_state=42)

# Second split: Validation (15%) and Test (15%)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp,
                                                test_size=0.50, random_state=42)

# Combine features + target for saving
train_set = pd.concat([X_train, y_train], axis=1)
val_set = pd.concat([X_val, y_val], axis=1)
test_set = pd.concat([X_test, y_test], axis=1)

# Save to CSV
train_set.to_csv("train.csv", index=False)
val_set.to_csv("val.csv", index=False)
test_set.to_csv("test.csv", index=False)
```

```
print("Train:", X_train.shape, y_train.shape)
print("Val:", X_val.shape, y_val.shape)
print("Test:", X_test.shape, y_test.shape)
```

Expected Output:

- Display of dataset head and shape.

	Company Names	Cars Names	Engines CC/Battery	Capacity	HorsePower
0	FERRARI	SF90 STRADALE	V8	3990 cc	963 hp
1	ROLLS ROYCE	PHANTOM	V12	6749 cc	563 hp
2	Ford	KA+	1.2L Petrol	1,200 cc	70-85 hp
3	MERCEDES	GT 63 S	V8	3,982 cc	630 hp
4	AUDI	AUDI R8 Gt	V10	5,204 cc	602 hp

- Shapes of train, validation, and test splits printed, e.g.:

Train: (525, 2) (525,)

Val: (112, 2) (112,)

Test: (113, 2) (113,)

- Three CSV files generated:

train.csv

val.csv

test.csv

Observations:

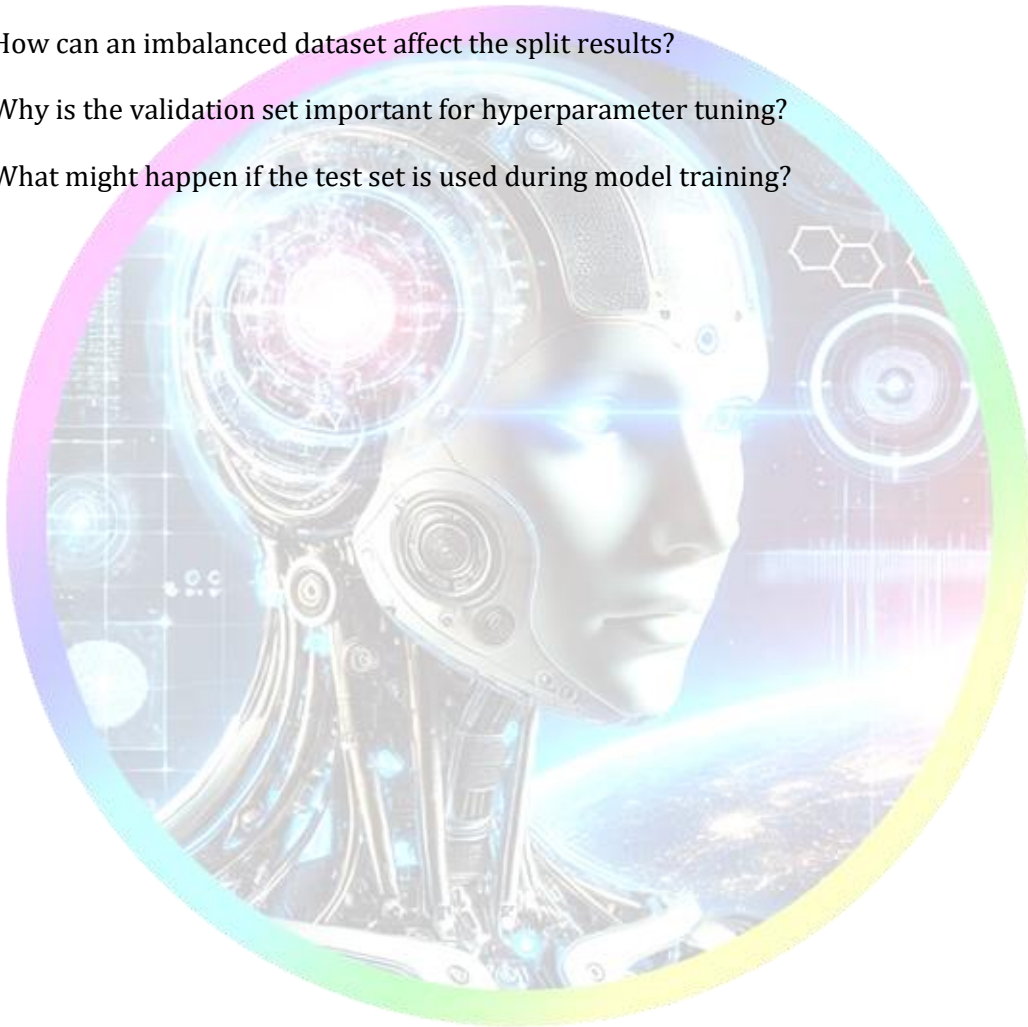
The dataset was successfully split into training, validation, and testing subsets in a 75-15-15 ratio. Each subset was saved into a separate CSV file for later use. This ensures model training, hyperparameter tuning, and evaluation are performed independently, reducing chances of overfitting.

Conclusion:

This experiment demonstrated the importance of splitting datasets into train, validation, and test subsets. Such partitioning allows for fair evaluation of machine learning models and ensures that final performance metrics are unbiased and reliable.

Viva Questions:

1. Why do we split the dataset into training, validation, and test sets?
2. What is the role of random_state in train_test_split?
3. How can an imbalanced dataset affect the split results?
4. Why is the validation set important for hyperparameter tuning?
5. What might happen if the test set is used during model training?



Experiment No: 03

Machine Learning

Aim:

Implement k-NN from scratch in Python to classify Iris flowers, varying k to observe accuracy changes.

Objective:

- To understand the working of the k-Nearest Neighbors (k-NN) algorithm.
- To implement the k-NN algorithm without using sklearn's built-in classifier.
- To evaluate model performance by varying the number of neighbors (k).
- To visualize the relationship between accuracy and k.

Prerequisites:

- Software: Python 3.x with numpy, matplotlib, scikit-learn.
- Hardware: A computer with at least 4GB RAM.
- Skills: Basic knowledge of Python and supervised learning.

Theory:

The k-Nearest Neighbors (k-NN) algorithm is a supervised machine learning method used for classification and regression.

It assigns the label of a new data point based on the majority vote of its k nearest neighbors in the training dataset.

The choice of k is crucial: a small k may lead to overfitting, while a large k may smooth decision boundaries excessively.

This experiment implements k-NN from scratch and analyzes accuracy variations for different values of k using the Iris dataset.

Apparatus/Software Requirements:

- Python IDE (Jupyter Notebook/VS Code).
- Internet connection (for library installation).
- Iris dataset (available in sklearn).

Procedure:

1. Step 1: Import required libraries (numpy, matplotlib, scikit-learn).
2. Step 2: Load the Iris dataset from sklearn.datasets.
3. Step 3: Split the dataset into training and testing sets (80-20 split).
4. Step 4: Implement the k-NN algorithm from scratch using Euclidean distance.
5. Step 5: Train the model with different values of k.
6. Step 6: Predict labels for the test set and compute accuracy.
7. Step 7: Plot accuracy vs k to analyze performance trends.

Program Code:

```
import numpy as np
from collections import Counter
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt

# -----
# KNN Implementation from Scratch
# -----
class KNN:
    def __init__(self, k=3):
        self.k = k

    def fit(self, X_train, y_train):
        self.X_train = X_train
        self.y_train = y_train

    def predict(self, X_test):
        predictions = [self._predict(x) for x in X_test]
        return np.array(predictions)

    def _predict(self, x):
        distances = [np.linalg.norm(x - x_train) for x_train in self.X_train]
        k_indices = np.argsort(distances)[:self.k]
        k_nearest_labels = [self.y_train[i] for i in k_indices]
        most_common = Counter(k_nearest_labels).most_common(1)
        return most_common[0][0]

# -----
# Load Data
# -----
iris = load_iris()
X, y = iris.data, iris.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# -----SSS
# Evaluate for different k
# -----
k_values = [1, 3, 5, 7, 9, 11, 15]
```

```

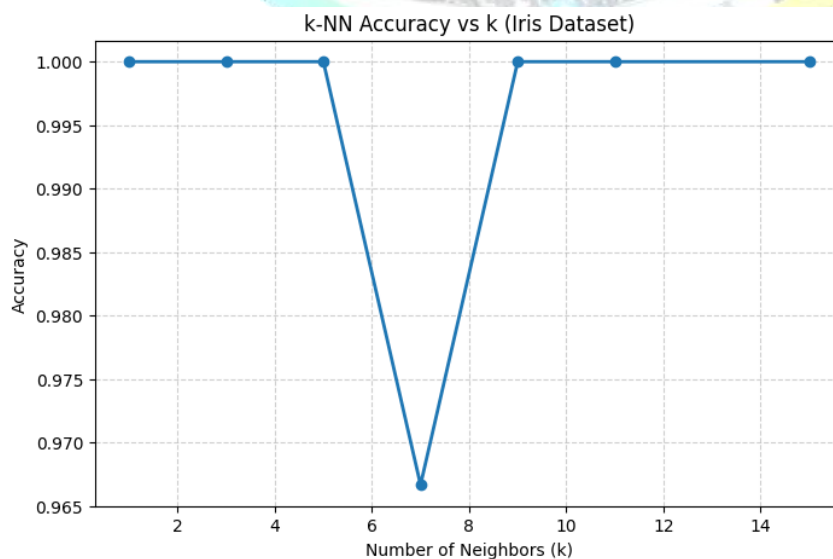
accuracies = []
for k in k_values:
    clf = KNN(k=k)
    clf.fit(X_train, y_train)
    predictions = clf.predict(X_test)
    accuracy = np.mean(predictions == y_test)
    accuracies.append(accuracy)
    print(f"k = {k}, Accuracy = {accuracy:.4f}")

# -----
# Plot Accuracy vs k
# -----
plt.figure(figsize=(8, 5))
plt.plot(k_values, accuracies, marker='o', linestyle='-', linewidth=2)
plt.title("k-NN Accuracy vs k (Iris Dataset)")
plt.xlabel("Number of Neighbors (k)")
plt.ylabel("Accuracy")
plt.grid(True, linestyle="--", alpha=0.6)
plt.show()

```

Expected Output:

- Accuracy values printed for different k values (1, 3, 5, 7, 9, 11, 15).
- k = 1, Accuracy = 1.0000
 k = 3, Accuracy = 1.0000
 k = 5, Accuracy = 1.0000
 k = 7, Accuracy = 0.9667
 k = 9, Accuracy = 1.0000
 k = 11, Accuracy = 1.0000
 k = 15, Accuracy = 1.0000
- A plot of accuracy vs k showing performance trends.



- Best accuracy obtained at an optimal value of k .

Observations:

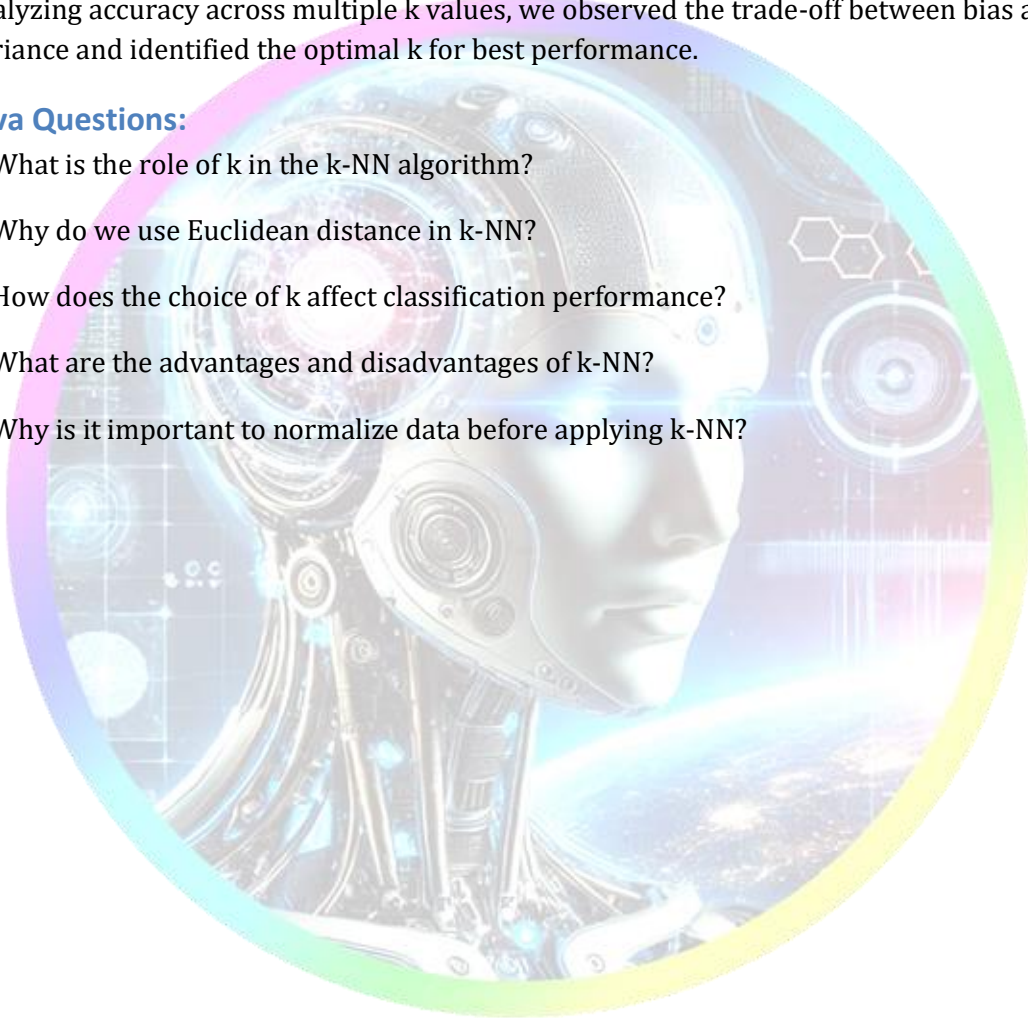
The accuracy of the k -NN classifier varied with different values of k . Smaller k values tended to overfit, while larger k values provided smoother but sometimes less accurate predictions. An optimal k value balanced bias and variance, giving the highest accuracy.

Conclusion:

This experiment implemented the k -NN algorithm from scratch on the Iris dataset. By analyzing accuracy across multiple k values, we observed the trade-off between bias and variance and identified the optimal k for best performance.

Viva Questions:

1. What is the role of k in the k -NN algorithm?
2. Why do we use Euclidean distance in k -NN?
3. How does the choice of k affect classification performance?
4. What are the advantages and disadvantages of k -NN?
5. Why is it important to normalize data before applying k -NN?



Experiment No: 04

Machine Learning

Aim:

Train a Linear Regression model on Boston Housing Data, visualizing residuals vs predictions.

Objective:

- To understand linear regression modeling on real-world datasets.
- To train a regression model using the Boston housing dataset.
- To visualize residuals and predictions for evaluating model performance.

Prerequisites:

- Software: Python 3.x with libraries: numpy, pandas, matplotlib, seaborn, scikit-learn.
- Hardware: A computer with at least 4GB RAM.
- Skills: Basic Python programming and knowledge of regression models.
- Dataset Availability: Boston Housing dataset (CSV format).

Theory:

Linear Regression is one of the simplest supervised machine learning algorithms used for predicting a dependent variable based on one or more independent variables. The Boston Housing dataset contains information on housing prices and several features such as crime rate, number of rooms, and distance to employment centers. By fitting a linear regression model, we can estimate housing prices and evaluate model performance using residuals (the difference between actual and predicted values). Visualizing residuals against predictions helps check model assumptions and detect patterns indicating poor fit or non-linearity.

Apparatus / Software Requirements:

- Python IDE or Jupyter Notebook
- Boston Housing dataset (CSV)
- Internet connection for installing libraries (if required)

Procedure:

Step 1: Import required libraries (numpy, pandas, matplotlib, seaborn, scikit-learn).

Step 2: Load the Boston Housing dataset into a DataFrame.

Step 3: Explore the dataset and visualize feature-target relationships.

Step 4: Split the data into training and testing sets.

Step 5: Train a Linear Regression model using training data.

Step 6: Evaluate model predictions using Mean Absolute Error (MAE).

Step 7: Visualize Actual vs Predicted values and plot residuals vs predictions.

Program Code:

Importing required libraries

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns

Importing scikit-learn modules

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_absolute_error

Load Boston Housing dataset

df = pd.read_csv("Boston_housing_data.csv")

df.head()

Selecting features and target

X = df.drop("medv", axis=1)

y = df["medv"]

Splitting the data into train and test sets

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

Initialize and train the Linear Regression model

model = LinearRegression()

model.fit(X_train, y_train)

Predicting on test data

y_pred = model.predict(X_test)

Mean Absolute Error

MAE = mean_absolute_error(y_pred, y_test)

print("Mean Absolute Error:", MAE)


```
# Plot Actual vs Predicted values
plt.scatter(y_test, y_pred, color="red")
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], color="black",
linewidth=4, label="Perfect Prediction")
plt.title("Actual vs Predicted Values")
plt.xlabel("Actual")
plt.ylabel("Predicted")
plt.legend()
plt.grid(True)
plt.show()
```

Expected Output:

1.

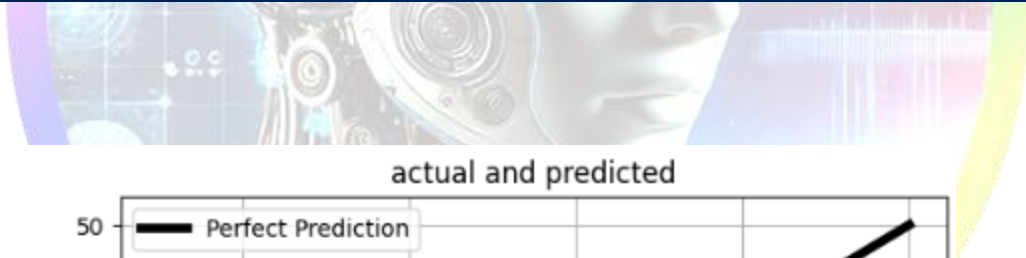
	Unnamed: 0	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	black	lstat	medv
0	1	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98	24.0
1	2	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14	21.6
2	3	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.7
3	4	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4
4	5	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33	36.2

2.

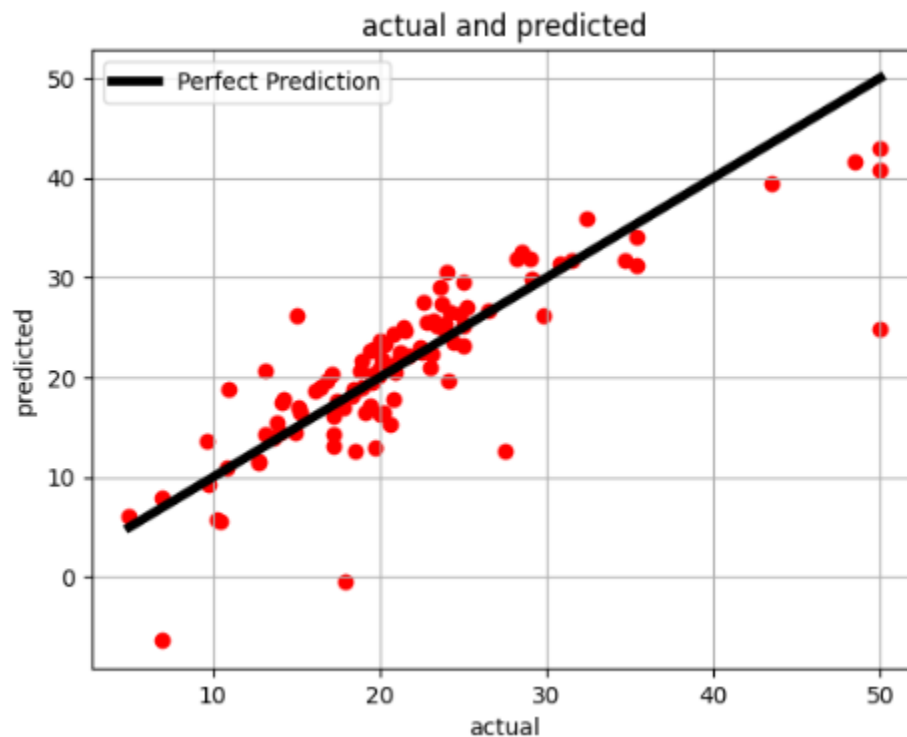
	Unnamed: 0	crim	zn	indus	chas	nox	rm	age	dis	\
477	478	15.02340	0.0	18.10	0	0.6140	5.304	97.3	2.1007	
15	16	0.62739	0.0	8.14	0	0.5380	5.834	56.5	4.4986	
332	333	0.03466	35.0	6.06	0	0.4379	6.031	23.3	6.6407	
423	424	7.05042	0.0	18.10	0	0.6140	6.103	85.1	2.0218	
19	20	0.72580	0.0	8.14	0	0.5380	5.727	69.5	3.7965	
..	...	...	...	...	...	...	...	...	...	
106	107	0.17120	0.0	8.56	0	0.5200	5.836	91.9	2.2110	
270	271	0.29916	20.0	6.96	0	0.4640	5.856	42.1	4.4290	
348	349	0.01501	80.0	2.01	0	0.4350	6.635	29.7	8.3440	
435	436	11.16040	0.0	18.10	0	0.7400	6.629	94.6	2.1247	
102	103	0.22876	0.0	8.56	0	0.5200	6.405	85.4	2.7147	

3.

```
array([28.97335364, 35.87795794, 13.97366183, 25.61583978, 18.66608153,  
23.64466022, 17.05971239, 14.5264485 , 22.79402759, 19.60618783,  
24.62043501, 19.06428675, -6.27197735, 22.20788474, 18.64849181,  
26.15022718, 20.62835191, 5.77374284, 40.72117979, 17.48530737,  
27.08436097, 29.90818637, 11.65330429, 22.99879937, 17.85048447,  
15.5556251 , 23.31650588, 14.44013189, 22.11106256, 18.20109763,  
22.33391113, 24.79774986, 26.15304018, 17.89425941, 16.52927207,  
17.10592481, 31.77372627, 19.58243073, 23.603617 , 25.2202457 ,  
12.89702838, 31.91216634, 42.90231353, 17.74616574, 27.5000776 ,  
17.06259552, 14.39460992, 26.51167547, 20.10908837, 30.51587021,  
21.74636797, 34.09117218, 16.44234996, 26.62572161, 39.42027423,  
22.49131562, 18.75511558, 32.62323592, 25.35325167, 12.58127149,  
23.23090959, 31.15654212, 31.70938821, 16.555899 , 19.75136599,  
16.30198537, 20.60703158, 26.4253367 , 31.3483927 , 11.44469483,  
20.93853272, 27.33689972, 10.88471601, 15.24992839, 24.31201729,  
6.16749235, 21.87268675, 41.62910729, 18.88336507, 7.99630902,  
21.29335313, 13.20061512, 20.55490914, 9.36676224, 22.6651137 ,  
31.82868525, 18.98610512, 25.13454836, 29.56561095, 20.35948068,  
25.6740408 , 5.63807591, 20.32530275, 14.25324163, 12.64785983,  
20.95671212, 24.91386093, -0.34555533, 13.62509538, 16.15367547,  
22.54865859, 25.00852603])
```



4.



Observations:

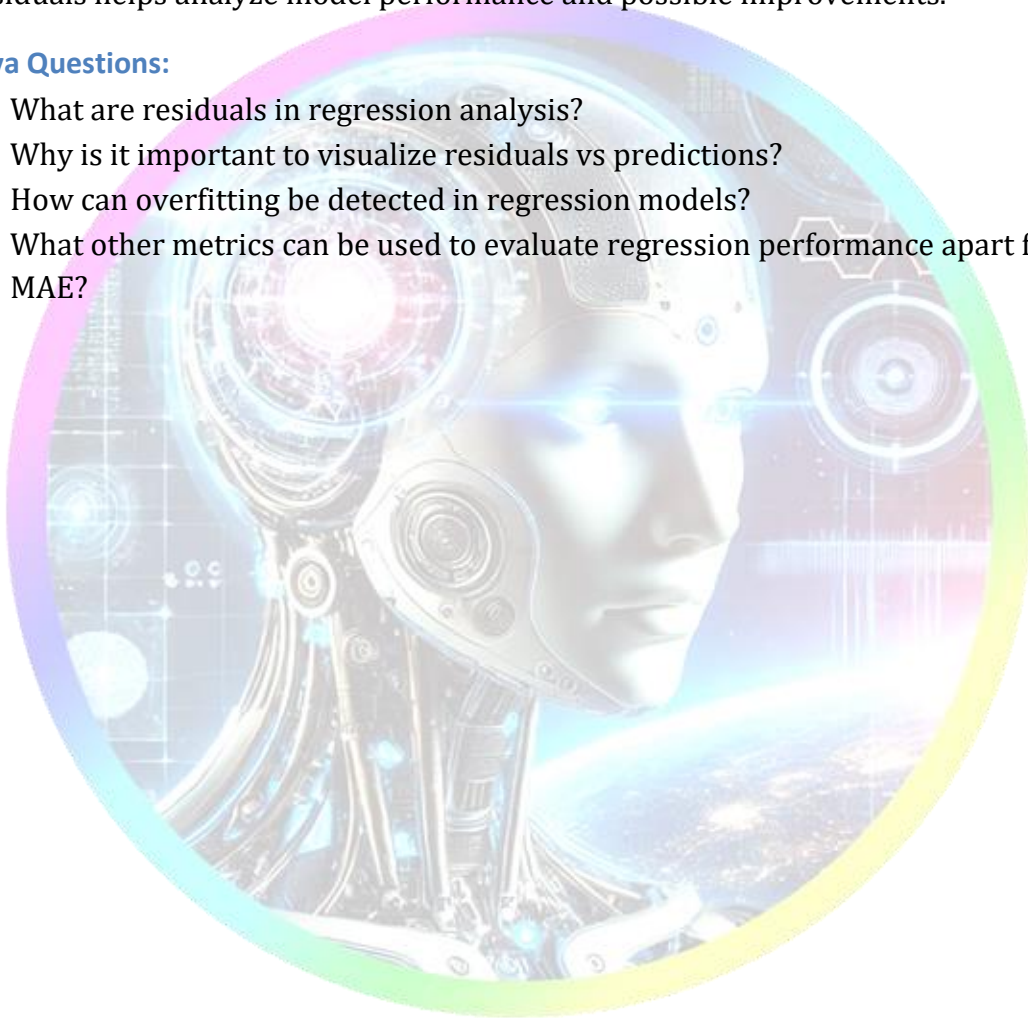
The linear regression model predicts housing prices with varying accuracy depending on the features. Residual plots help assess whether the model assumptions hold and highlight any non-linearity or heteroscedasticity.

Conclusion:

This experiment demonstrates the application of Linear Regression on the Boston Housing dataset. The model provides reasonable predictions, and visualization of residuals helps analyze model performance and possible improvements.

Viva Questions:

- What are residuals in regression analysis?
- Why is it important to visualize residuals vs predictions?
- How can overfitting be detected in regression models?
- What other metrics can be used to evaluate regression performance apart from MAE?



Experiment No: 05

Machine Learning

Aim:

Build a logistic regression classifier for spam detection, evaluating with precision-recall curves.

Objective:

- To understand the use of Logistic Regression for text classification.
- To learn feature extraction using TF-IDF for SMS spam dataset.
- To evaluate classification models using Precision-Recall curves and Average Precision.
- To analyze confusion matrix and classification report for performance assessment.

Prerequisites:

- Software: Python 3.x installed with libraries: pandas, scikit-learn, matplotlib.
- Hardware: A computer with at least 4 GB RAM.
- Skills: Basic Python programming, ML basics, handling datasets.

Theory:

Logistic Regression is a linear model commonly used for binary classification tasks. It estimates the probability that a given input belongs to a particular class using the sigmoid function.

In text classification, feature extraction plays a key role. The Term Frequency-Inverse Document Frequency (TF-IDF) method converts text into numerical feature vectors.

Precision-Recall curves are useful evaluation tools for imbalanced datasets. They show the trade-off between precision and recall for different thresholds. Average Precision (AP) summarizes the curve in a single value.

Apparatus/Software Requirements:

- Python IDE (Jupyter Notebook or any editor).
- Required Python libraries: pandas, scikit-learn, matplotlib.
- SMS Spam Dataset (label + message format).

Procedure:

Step 1: Load SMS Spam dataset and inspect structure.

Step 2: Preprocess labels (ham=0, spam=1).

Step 3: Split dataset into training and testing sets.

Step 4: Transform messages using TF-IDF Vectorizer.

Step 5: Train Logistic Regression classifier.

Step 6: Evaluate model using Precision-Recall curves and Average Precision.

Step 7: Generate confusion matrix and classification report.

Program Code:

```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import precision_recall_curve, average_precision_score
import matplotlib.pyplot as plt

# Load dataset
df = pd.read_csv('sms_spam_dataset.csv', sep='\t', header=None, names=['label',
'message'])
df['label_num'] = df['label'].map({'ham': 0, 'spam': 1})

# Split data
X_train, X_test, y_train, y_test = train_test_split(df['message'], df['label_num'],
test_size=0.2, random_state=42, stratify=df['label_num'])

# TF-IDF feature extraction
vectorizer = TfidfVectorizer(stop_words='english')
X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)

# Logistic Regression model
clf = LogisticRegression(max_iter=1000)
clf.fit(X_train_tfidf, y_train)

# Evaluate with PR Curve
y_scores = clf.predict_proba(X_test_tfidf)[:, 1]
precision, recall, thresholds = precision_recall_curve(y_test, y_scores)
avg_precision = average_precision_score(y_test, y_scores)

# Plot PR Curve
plt.figure(figsize=(8,6))
plt.plot(recall, precision, label=f'LogReg (AP={avg_precision:.2f})')
```



```
plt.xlabel("Recall")
plt.ylabel("Precision")
plt.title("Precision-Recall Curve (Spam Detection)")
plt.legend()
plt.grid(True)
plt.show()
```

Confusion Matrix & Report

```
print(confusion_matrix(y_test, clf.predict(X_test_tfidf)))
print(classification_report(y_test, clf.predict(X_test_tfidf),
target_names=['ham', 'spam']))
```

Expected Output:

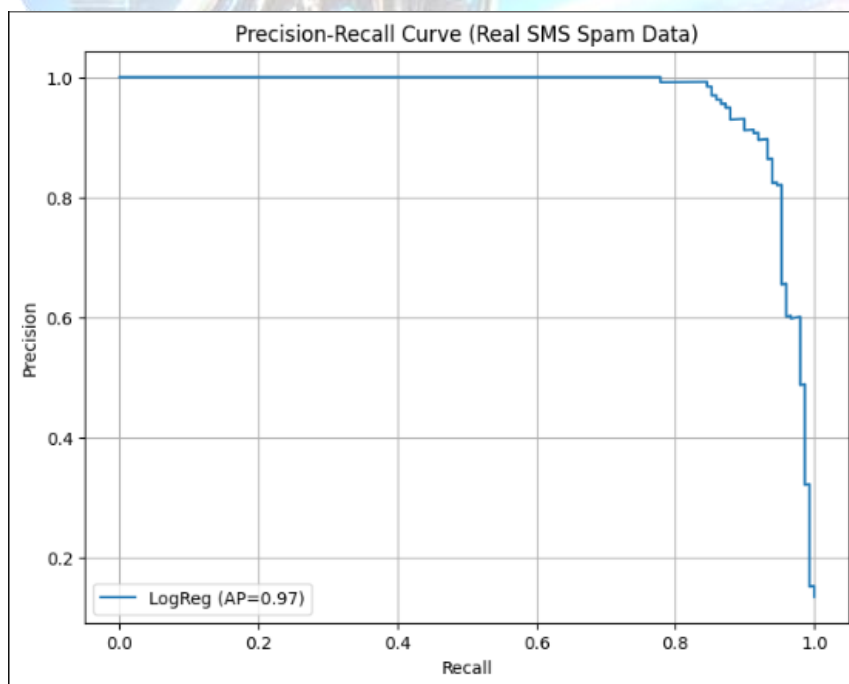
- Display dataset shape and distribution of ham/spam.

```
(5572, 2)
```

- Quick peek. (head)

```
label      message
0  ham  Go until jurong point, crazy.. Available only ...
1  ham                Ok lar... Joking wif u oni...
2  spam  Free entry in 2 a wkly comp to win FA Cup fina...
3  ham  U dun say so early hor... U c already then say...
4  ham  Nah I don't think he goes to usf, he lives aro...
label
ham    0.865937
spam   0.134063
Name: proportion, dtype: float64
```

- Precision-Recall curve plotted with Average Precision displayed in legend.



Observations:

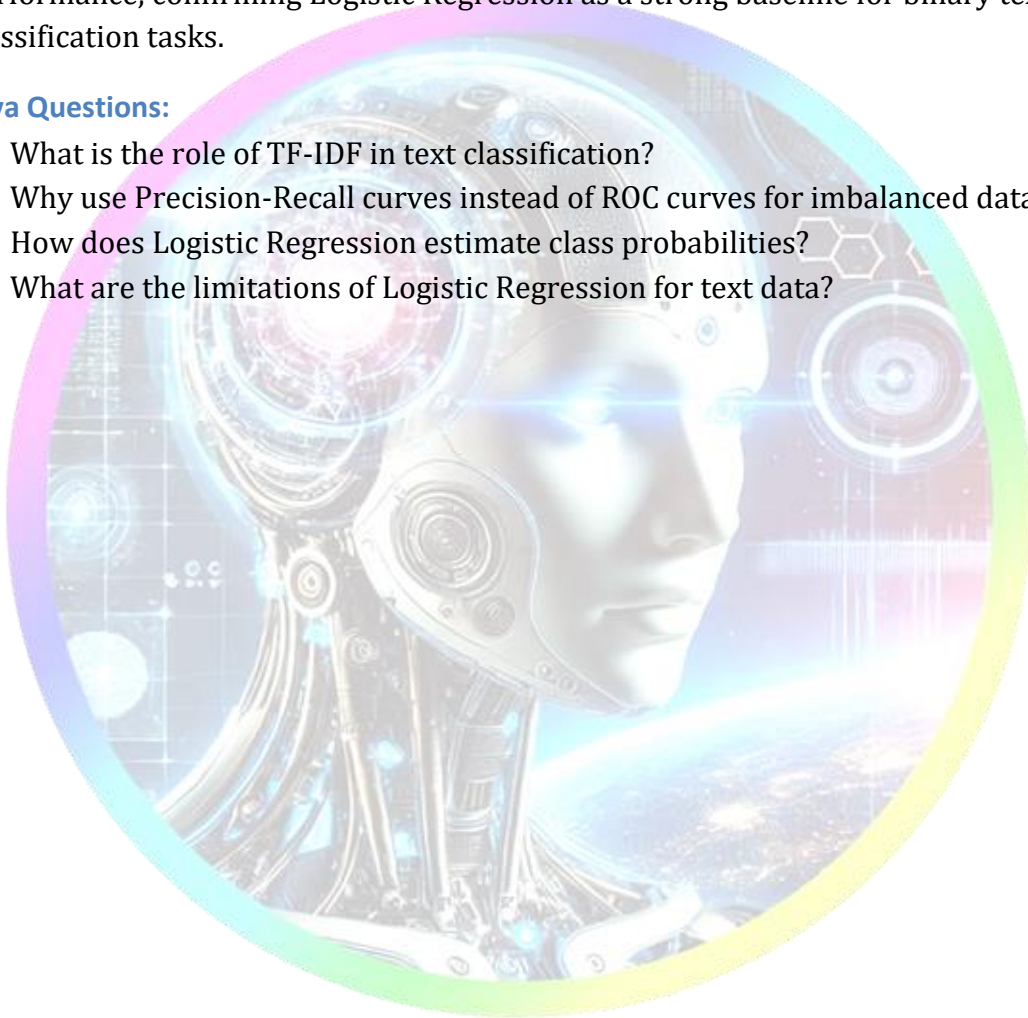
The Logistic Regression classifier achieves high performance in distinguishing spam from ham messages. The Precision-Recall curve indicates model stability even under class imbalance. Confusion matrix and report provide detailed error analysis.

Conclusion:

This lab demonstrated spam detection using Logistic Regression with TF-IDF feature extraction. Evaluation with Precision-Recall curves showed robust performance, confirming Logistic Regression as a strong baseline for binary text classification tasks.

Viva Questions:

- What is the role of TF-IDF in text classification?
- Why use Precision-Recall curves instead of ROC curves for imbalanced datasets?
- How does Logistic Regression estimate class probabilities?
- What are the limitations of Logistic Regression for text data?



Experiment No: 06

Machine Learning

Aim:

Optimize SVM hyperparameters (C, kernel) via grid search on a synthetic 2D dataset.

Objective:

- To understand the working of Support Vector Machines (SVM) for classification tasks.
- To learn how hyperparameters like C and kernel affect SVM model performance.
- To implement hyperparameter optimization using GridSearchCV.
- To analyze confusion matrix and classification report for performance evaluation.

Prerequisites:

- Software: Python 3.x with libraries: pandas, scikit-learn, numpy.
- Hardware: A system with at least 4 GB RAM.
- Skills: Basic Python programming and understanding of SVMs and model evaluation.

Theory:

Support Vector Machines (SVM) are powerful supervised learning models used for classification and regression tasks. They work by finding the optimal hyperplane that separates data points of different classes with the maximum margin.

The performance of an SVM largely depends on its hyperparameters:

- The regularization parameter (C) controls the trade-off between maximizing the margin and minimizing classification errors.
- The kernel function (linear, polynomial, RBF, etc.) maps the input data into higher dimensions, making it possible to handle non-linear relationships.

Hyperparameter tuning is crucial for achieving optimal model performance. Grid Search systematically explores a predefined set of hyperparameter combinations and selects the one that yields the best cross-validation score.

Apparatus/Software Requirements:

- Python IDE (Jupyter Notebook, VS Code, etc.).
- Required Python libraries: pandas, scikit-learn, numpy.
- A synthetic 2D dataset for visualization and model training.

Procedure:

Step 1: Load or generate a synthetic 2D dataset and inspect its structure.

Step 2: Preprocess the data (scaling and encoding if required).

Step 3: Split the dataset into training and testing sets.

Step 4: Define an SVM model and parameter grid for C, kernel, and gamma values.

Step 5: Perform GridSearchCV to identify the best hyperparameter combination.

Step 6: Evaluate the best model using the test dataset.

Step 7: Display confusion matrix and classification report for model performance.

Program Code:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.svm import SVC
from sklearn.metrics import classification_report, confusion_matrix
```

1. Load dataset

```
df = pd.read_csv("synthetic_dataset.csv")
print("Dataset Shape:", df.shape)
print(df.head())
```

2. Define features and target

```
X = df.drop(columns=["target"], errors="ignore")
y = df["target"]
```

3. Encode target if necessary

```
le = LabelEncoder()
y = le.fit_transform(y)
```

4. Train-test split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

5. Scale features

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```


6. SVM + GridSearchCV

```
param_grid = {  
    "C": [0.1, 1, 10],  
    "kernel": ["linear", "rbf", "poly"],  
    "gamma": ["scale", "auto"]  
}  
svm = SVC()  
grid = GridSearchCV(svm, param_grid, cv=3, scoring="accuracy", n_jobs=-1)  
grid.fit(X_train, y_train)
```

7. Results

```
print("Best Parameters:", grid.best_params_)  
print("Best CV Accuracy:", grid.best_score_)
```

8. Test set evaluation

```
best_model = grid.best_estimator_  
y_pred = best_model.predict(X_test)  
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))  
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

Expected Output:

- Display dataset shape and first few rows.

```
Dataset Shape: (569, 32)  
   id diagnosis  radius_mean  texture_mean  perimeter_mean  area_mean  \  
0   842302      M      17.99      10.38      122.80      1001.0  
1   842517      M      20.57      17.77      132.90      1326.0  
2  84300903      M      19.69      21.25      130.00      1203.0  
3  84348301      M      11.42      20.38       77.58       386.1  
4  84358402      M      20.29      14.34      135.10      1297.0  
  
   smoothness_mean  compactness_mean  concavity_mean  concave_points_mean  \  
0       0.11840      0.27760      0.3001      0.14710  
1       0.08474      0.07864      0.0869      0.07017  
2       0.10960      0.15990      0.1974      0.12790  
3       0.14250      0.28390      0.2414      0.10520  
4       0.10030      0.13280      0.1980      0.10430  
  
   ... radius_worst  texture_worst  perimeter_worst  area_worst  \  
0   ...      25.38      17.33      184.60      2019.0  
1   ...      24.99      23.41      158.80      1956.0  
2   ...      23.57      25.53      152.50      1709.0  
3   ...      14.91      26.50       98.87       567.7  
4   ...      22.54      16.67      152.20      1575.0
```


- Best hyperparameters and corresponding cross-validation accuracy.

```
Best Parameters: {'C': 0.1, 'gamma': 'scale', 'kernel': 'linear'}  
Best CV Accuracy: 0.9780411293133495
```

- Confusion matrix and classification report.

```
Confusion Matrix:  
[[72  0]  
 [ 2 40]]  
  
Classification Report:  
              precision    recall  f1-score   support  
  
     0       0.97        1.00        0.99         72  
     1       1.00        0.95        0.98         42  
  
 accuracy          0.98         0.98         0.98        114  
 macro avg          0.99         0.98         0.98        114  
 weighted avg       0.98         0.98         0.98        114
```

Observations:

The optimized SVM model shows improved accuracy after hyperparameter tuning. Grid Search helps systematically identify the best parameter set for optimal model performance.

Conclusion:

This experiment demonstrated how hyperparameter optimization using GridSearchCV enhances the performance of Support Vector Machines. The process provided insights into the influence of C, kernel, and gamma on model generalization.

Viva Questions:

- What does the parameter C control in SVM?
- What is the role of kernel functions in SVM?
- Why is feature scaling important before training an SVM?
- What is the purpose of GridSearchCV?
- How can we handle overfitting in SVM models?

Experiment No: 07

Machine Learning

Aim:

Compute confusion matrices and F1 score for a diabetes prediction model (Pima dataset).

Objective:

- To apply Logistic Regression for binary classification on medical data.
- To evaluate the performance of a model using confusion matrix and F1 score.
- To visualize model results using a confusion matrix heatmap.
- To interpret performance metrics like precision, recall, and F1 score.

Prerequisites:

- Software: Python 3.x with pandas, scikit-learn, seaborn, matplotlib installed.
- Hardware: A computer with at least 4 GB RAM.
- Skills: Understanding of Logistic Regression and evaluation metrics in ML.

Theory:

In classification tasks, the confusion matrix is a crucial tool to evaluate model performance. It displays the counts of true positives, true negatives, false positives, and false negatives. From these values, metrics such as precision, recall, and F1 score are derived.

The F1 score is the harmonic mean of precision and recall, providing a balance between the two. It is particularly useful when dealing with imbalanced datasets.

In this experiment, Logistic Regression is applied on the Pima diabetes dataset. This model estimates the probability of a patient being diabetic based on medical attributes. Feature scaling ensures numerical stability and helps the model converge effectively.

Apparatus/Software Requirements:

- Python IDE (Jupyter Notebook, VS Code, etc.).
- Required Python libraries: pandas, numpy, scikit-learn, seaborn, matplotlib.
- Pima Diabetes Dataset (CSV format).

Procedure:

Step 1: Load the Pima Diabetes dataset and inspect the structure.

Step 2: Split features and target variable (Outcome).

Step 3: Divide the data into training and testing sets.

Step 4: Apply feature scaling to improve model performance.

Step 5: Train Logistic Regression model on the training data.

Step 6: Predict outcomes on the test data.

Step 7: Compute confusion matrix and F1 score for evaluation.

Step 8: Plot the confusion matrix as a heatmap using seaborn.

Step 9: Print classification report for detailed analysis.

Program Code:

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, f1_score, classification_report

# Load dataset
df = pd.read_csv("Pima_diabetes.csv") # Replace with your actual dataset path

# Split features and target
X = df.drop('Outcome', axis=1)
y = df['Outcome']

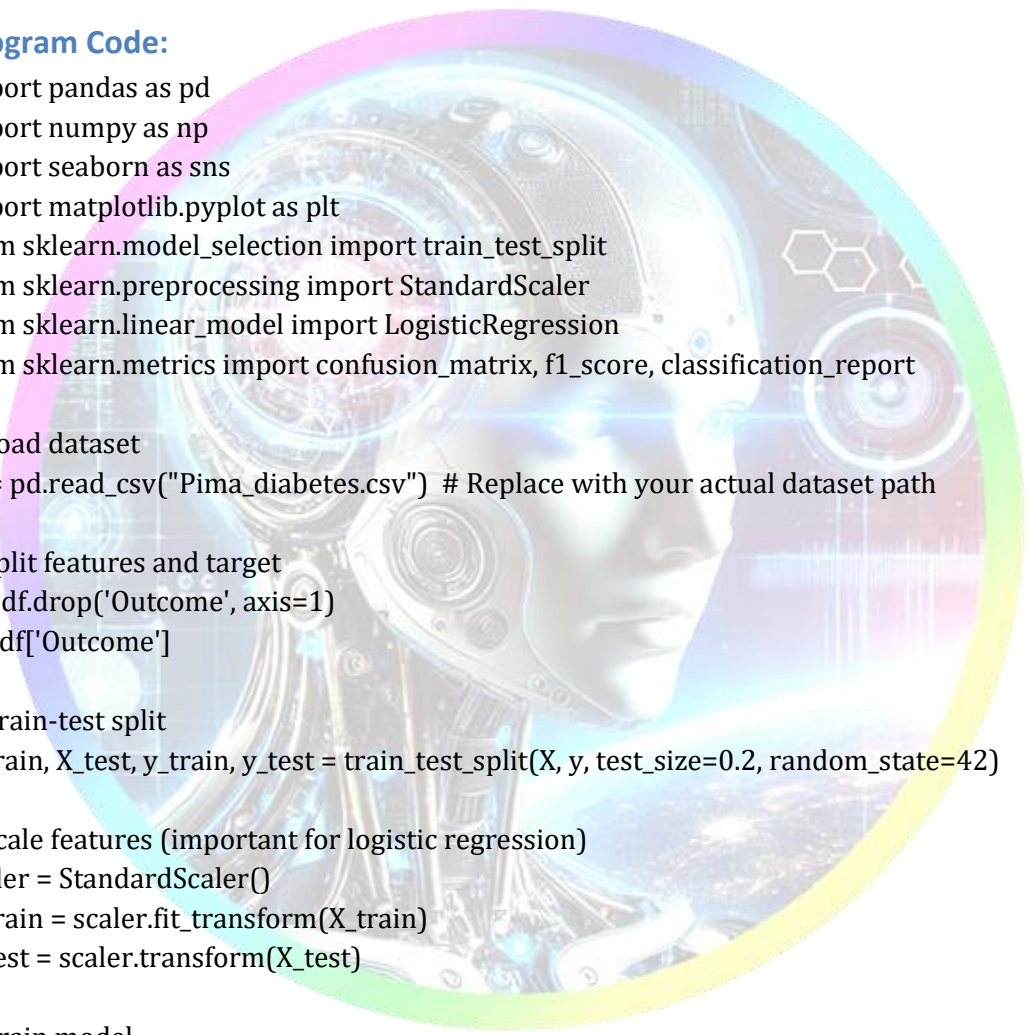
# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Scale features (important for logistic regression)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train model
model = LogisticRegression()
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Metrics
cm = confusion_matrix(y_test, y_pred)
```



```

f1 = f1_score(y_test, y_pred)
print("Confusion Matrix:\n", cm)
print("F1 Score:", f1)

# Normalized confusion matrix (percentages)
cm_percent = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis] * 100

# Plot confusion matrix
plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt='d', cmap='cool', cbar=False, square=True,
            xticklabels=['Not Diabetic (0)', 'Diabetic (1)'],
            yticklabels=['Not Diabetic (0)', 'Diabetic (1)'])
plt.title(f'Confusion Matrix (F1 Score = {f1:.2f})', fontsize=13)
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()

# Print classification report
print("Classification Report:\n", classification_report(y_test, y_pred))

```

Expected Output:

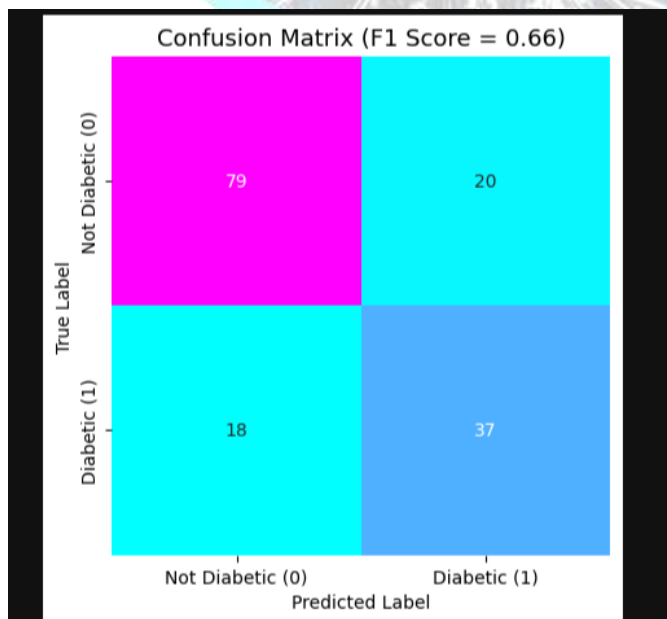
- Confusion Matrix displaying predicted vs actual outcomes & F1 Score printed on the console.

```

Confusion Matrix:
[[79 20]
 [18 37]]
F1 Score: 0.6607142857142857

```

- Heatmap visualization of the confusion matrix.



- Classification report showing precision, recall, and F1 for each class.

Classification Report:				
	precision	recall	f1-score	support
0	0.81	0.80	0.81	99
1	0.65	0.67	0.66	55
accuracy			0.75	154
macro avg	0.73	0.74	0.73	154
weighted avg	0.76	0.75	0.75	154

Observations:

The Logistic Regression model effectively classifies diabetic and non-diabetic cases. The confusion matrix shows correct and incorrect predictions, while the F1 score summarizes the model's overall performance. Visualization helps in understanding class-wise accuracy and misclassification rates.

Conclusion:

This experiment demonstrates model evaluation using confusion matrix and F1 score. These metrics provide comprehensive insights into classifier performance, especially in medical datasets like Pima Diabetes.

Viva Questions:

- What is a confusion matrix?
- How is the F1 score calculated?
- Why is feature scaling important in Logistic Regression?
- What does a high F1 score indicate?
- How can the confusion matrix be used to identify model weaknesses?

Experiment No: 07

Machine Learning

Aim:

Compute confusion matrices and F1 score for a diabetes prediction model (Pima dataset).

Objective:

- To apply Logistic Regression for binary classification on medical data.
- To evaluate the performance of a model using confusion matrix and F1 score.
- To visualize model results using a confusion matrix heatmap.
- To interpret performance metrics like precision, recall, and F1 score.

Prerequisites:

- Software: Python 3.x with pandas, scikit-learn, seaborn, matplotlib installed.
- Hardware: A computer with at least 4 GB RAM.
- Skills: Understanding of Logistic Regression and evaluation metrics in ML.

Theory:

In classification tasks, the confusion matrix is a crucial tool to evaluate model performance. It displays the counts of true positives, true negatives, false positives, and false negatives. From these values, metrics such as precision, recall, and F1 score are derived.

The F1 score is the harmonic mean of precision and recall, providing a balance between the two. It is particularly useful when dealing with imbalanced datasets.

In this experiment, Logistic Regression is applied on the Pima diabetes dataset. This model estimates the probability of a patient being diabetic based on medical attributes. Feature scaling ensures numerical stability and helps the model converge effectively.

Apparatus/Software Requirements:

- Python IDE (Jupyter Notebook, VS Code, etc.).
- Required Python libraries: pandas, numpy, scikit-learn, seaborn, matplotlib.
- Pima Diabetes Dataset (CSV format).

Procedure:

Step 1: Load the Pima Diabetes dataset and inspect the structure.

Step 2: Split features and target variable (Outcome).

Step 3: Divide the data into training and testing sets.

Step 4: Apply feature scaling to improve model performance.

Step 5: Train Logistic Regression model on the training data.

Step 6: Predict outcomes on the test data.

Step 7: Compute confusion matrix and F1 score for evaluation.

Step 8: Plot the confusion matrix as a heatmap using seaborn.

Step 9: Print classification report for detailed analysis.

Program Code:

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, f1_score, classification_report

# Load dataset
df = pd.read_csv("Pima_diabetes.csv") # Replace with your actual dataset path

# Split features and target
X = df.drop('Outcome', axis=1)
y = df['Outcome']

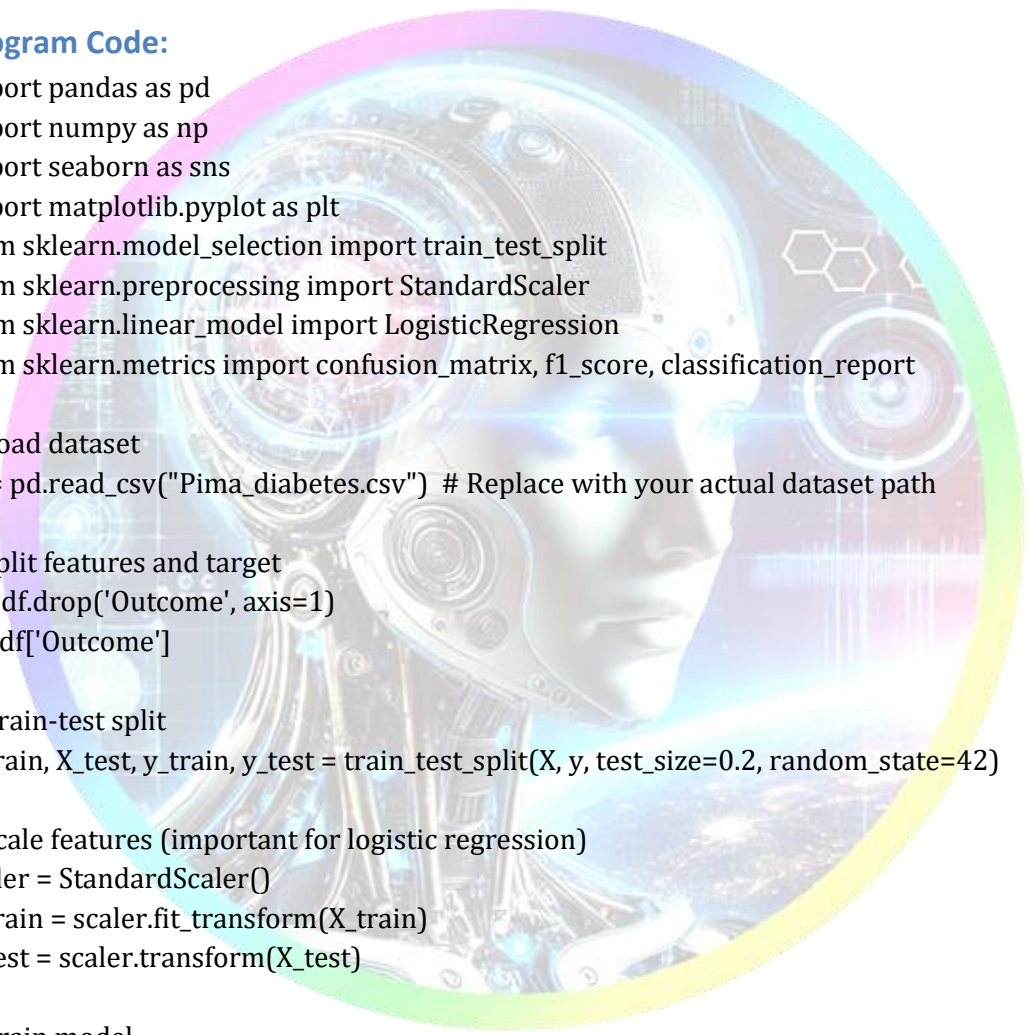
# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Scale features (important for logistic regression)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train model
model = LogisticRegression()
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Metrics
cm = confusion_matrix(y_test, y_pred)
```



```
f1 = f1_score(y_test, y_pred)
print("Confusion Matrix:\n", cm)
print("F1 Score:", f1)

# Normalized confusion matrix (percentages)
cm_percent = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis] * 100

# Plot confusion matrix
plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt='d', cmap='cool', cbar=False, square=True,
            xticklabels=['Not Diabetic (0)', 'Diabetic (1)'],
            yticklabels=['Not Diabetic (0)', 'Diabetic (1)'])
plt.title(f'Confusion Matrix (F1 Score = {f1:.2f})', fontsize=13)
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()

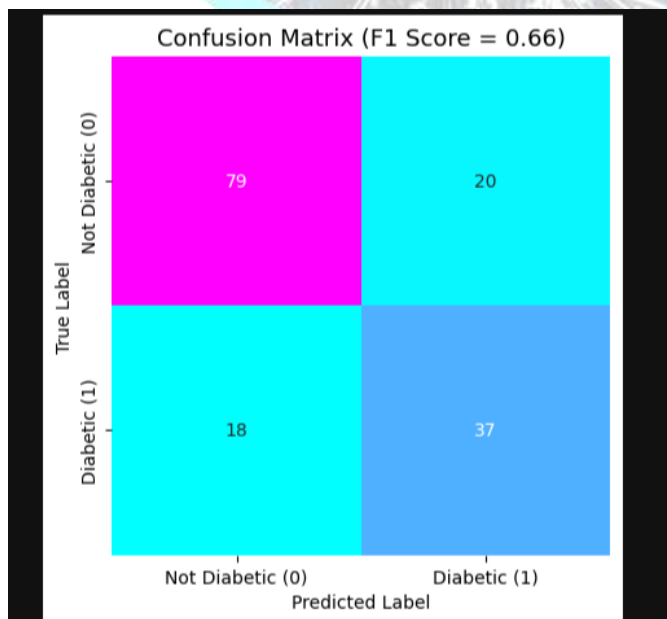
# Print classification report
print("Classification Report:\n", classification_report(y_test, y_pred))
```

Expected Output:

- Confusion Matrix displaying predicted vs actual outcomes & F1 Score printed on the console.

```
Confusion Matrix:
[[79 20]
 [18 37]]
F1 Score: 0.6607142857142857
```

- Heatmap visualization of the confusion matrix.



- Classification report showing precision, recall, and F1 for each class.

Classification Report:				
	precision	recall	f1-score	support
0	0.81	0.80	0.81	99
1	0.65	0.67	0.66	55
accuracy			0.75	154
macro avg	0.73	0.74	0.73	154
weighted avg	0.76	0.75	0.75	154

Observations:

The Logistic Regression model effectively classifies diabetic and non-diabetic cases. The confusion matrix shows correct and incorrect predictions, while the F1 score summarizes the model's overall performance. Visualization helps in understanding class-wise accuracy and misclassification rates.

Conclusion:

This experiment demonstrates model evaluation using confusion matrix and F1 score. These metrics provide comprehensive insights into classifier performance, especially in medical datasets like Pima Diabetes.

Viva Questions:

- What is a confusion matrix?
- How is the F1 score calculated?
- Why is feature scaling important in Logistic Regression?
- What does a high F1 score indicate?
- How can the confusion matrix be used to identify model weaknesses?